Sesiones de refuerzo: programación y lógica

Carlos A Delgado S
Pontificia Universidad Javeriana Cali

De qué se trata

Dieciséis horas de refuerzo repartidas en cuatro sábados de cuatro horas. Desde el segundo, cada sábado dedica tres horas a programación y una a lógica.

Los conceptos de programación se dan por vistos en Estructuras de Datos, así que el repaso es práctico. Cada hora de programación gira alrededor de un problema de juez en línea del nivel de las tareas de ese curso, y la estructura o la idea que hace falta se recuerda en el momento en que el problema la pide, con lo que cuesta cada operación. Ninguna hora expone una estructura entera. Cada problema cierra con su costo: de qué depende el tamaño de la entrada, cuántas veces corre cada ciclo y el total en O.

La línea de programación va de arreglos y matrices a la bisección y los recorridos en grafos, y termina en colas de prioridad, bisección sobre la respuesta y dividir y vencerás. La de lógica repasa lo que hace falta para leer y escribir una demostración: predicados y cuantificadores, los tipos de demostración, la cota con O grande y, en dos horas, el invariante de ciclo, planteado sobre los ciclos de los problemas de clase.

Al terminar, el estudiante elige la estructura que un problema pide, sabe cuánto le cuesta cada operación y escribe el invariante de un ciclo como fórmula. Los temas del curso dan esa base por vista.

Día	Hora	Tema	Problema en clase
17 de octubre	1	Estructuras lineales y su costo	
	2	Tablas hash, árboles balanceados y montículos	
	3	Memoria y pila de llamadas: el factorial en el depurador	
	4	Arreglos: un recorrido contra dos	UVa 11078, Open Credit System
24 de octubre	1	Acumulados y el mejor tramo	UVa 507, Jill Rides Again
	2	UVa 507 en el computador: lectura, numeración de paradas y desempate	UVa 507, Jill Rides Again
	3	Acumulados en dos dimensiones; fijar dos filas y reducir al mejor tramo	UVa 108, Maximum Sum
	4	Lógica: proposiciones, cuantificadores y la definición de O	en papel
7 de noviembre	1	Bisección: encontrar la función	UVa 12032, The Monkey and the Oiled Bamboo
	2	Recorrido en amplitud con tope de pasos	UVa 336, A Node Too Far
	3	Recorrido en profundidad y ciclos en un grafo dirigido	UVa 11686, Pick up sticks
	4	Lógica: invariantes de ciclo I	en papel
14 de noviembre	1	Colas de prioridad	UVa 1203, Argus
	2	Bisección: la función dada y sobre la respuesta	UVa 10341, Solve It; UVa 11413, Fill the Containers
	3	Dividir y vencerás: de dónde sale la recursión	UVa 10810, Ultra-QuickSort
	4	Lógica: tipos de demostración e invariantes de ciclo II	en papel

Horarios

Las sesiones son los sábados 17 y 24 de octubre y 7 y 14 de noviembre. El 31 de octubre no hay sesión por el parcial, así que entre el 24 de octubre y el 7 de noviembre pasan dos semanas.

Cómo se trabaja una sesión

- Cada hora tiene tres tramos. Los primeros diez a veinte minutos explican la idea con un ejemplo pequeño y presentan el ejercicio. El resto de la hora, menos los últimos diez minutos, se resuelve el ejercicio: en el computador las horas de programación, en papel la de lógica. Al final se muestra la solución con su costo.
- Las diapositivas de la clase llevan la explicación y el enunciado; la solución va en un documento aparte.
- Los lenguajes son Python y C++. Quien viene de Estructuras de Datos puede resolver el mismo ejercicio en C.
- Solo biblioteca estándar, sin `break`, sin `continue` y con un solo `return` al final de cada función.
- La hora de lógica se trabaja en papel, y sus ejemplos salen de los problemas resueltos ese día.
- Las sesiones del 24 de octubre y del 7 y 14 de noviembre dejan tres problemas de tarea cada una. Se resuelven antes de la sesión siguiente, se envían al juez y se revisan en vivo en sus primeros minutos.

Programación por horas

El 17 de octubre fue un repaso de Estructuras de Datos con su costo y su implementación, y cerró con UVa 11078. El 24 de octubre UVa 507 ocupó dos horas, así que la primera hora de invariantes pasó al 7 de noviembre y la hora de lógica fue la del 17 de octubre, que ese día no se dio. La recursión sobre un árbol, las colas y los diccionarios quedan como tarea y como problemas propuestos, y UVa 924 queda como problema propuesto para seguir con el recorrido en amplitud. Las sesiones del 7 y el 14 de noviembre empiezan con la revisión de la tarea de la sesión anterior.

Hora	Lo que el problema pide	Problema en el computador
Sábado 17 de octubre. Estructuras y un recorrido		
1	Arreglo dinámico y lista, lista enlazada, deque, pila y cola, con lo que cuesta cada operación.	
2	Direccionamiento directo y tabla hash, árbol balanceado y montículo, con lo que cuesta cada operación.	
3	Iteradores y memoria; la pila de llamadas del factorial seguida en el depurador.	
4	Acceso por posición en $O(1)$; cambiar un doble ciclo de $\Theta(n^2)$ por un recorrido que lleva lo mejor visto hasta ahí.	UVa 11078, *Open Credit System*: la mayor diferencia $a_i - a_j$ con $i < j$.
Sábado 24 de octubre. Tramos, matrices y lógica		
1	Acumulados de un arreglo; el mejor tramo que termina en cada posición y cómo sale del anterior.	UVa 507, *Jill Rides Again*: el tramo de ruta de mayor suma.
2	La lectura con `scanf`, la numeración de las paradas desde 1 y el desempate por longitud y por inicio.	UVa 507, *Jill Rides Again*: la solución completa en el computador.
3	Acumulados en dos dimensiones; con dos filas fijas, las sumas por columna forman un arreglo.	UVa 108, *Maximum Sum*: fijar dos filas y reducir al mejor tramo.
4	Lógica, en papel: proposiciones, predicados y cuantificadores para expresar propiedades, y la definición de O leída como fórmula.	
Sábado 7 de noviembre. Bisección y recorridos en grafos		
1	Revisión de la tarea. De pedir el óptimo a comprobar un candidato; las pistas que delatan la función de decisión y por qué hay que verificar la monotonía.	UVa 12032, *The Monkey and the Oiled Bamboo*: la menor fuerza inicial con que se sube la escalera.
2	Cola y arreglo de distancias; los nodos salen por distancia creciente, así que el tope se aplica al expandir; diccionario de número a índice.	UVa 336, *A Node Too Far*: cuántos nodos quedan por fuera con un TTL.
3	Los tres estados de un nodo: sin visitar, en camino y terminado; qué arista cierra un ciclo y por qué el orden de terminación al revés sirve.	UVa 11686, *Pick up sticks*: un orden de recogida o `IMPOSSIBLE`.

Hora	Lo que el problema pide	Problema en el computador
4	Lógica, en papel: invariantes de ciclo escritos como fórmula; *mayor* de UVa 11078 con inicialización, estabilidad y terminación, el invariante de *mejor* y por qué depende de *mayor*; UVa 507 sin el desempate como ejercicio.	
Sábado 14 de noviembre. Colas de prioridad, bisección y dividir y vencerás		
1	Revisión de la tarea. Insertar y extraer en una cola de prioridad en $O(\log n)$, con pares: `heapq` con tuplas en Python, `priority_queue` con comparador en C++.	UVa 1203, *Argus*: las primeras K respuestas de un grupo de consultas periódicas.
2	La función ya dada: la monotonía se comprueba derivando y los extremos deciden si hay raíz. Después, la primera posición que cumple una condición en $[l, r)$ y, de optimización a decisión, una pregunta monótona y la bisección sobre la respuesta.	UVa 10341, *Solve It*: la raíz de una función en $[0, 1]$. UVa 11413, *Fill the Containers*: la menor capacidad máxima de m contenedores.
3	Las parejas fuera de orden se reparten en tres clases: las de cada mitad son el mismo problema más chico, de ahí la recursión, y las que cruzan se cuentan al mezclar.	UVa 10810, *Ultra-QuickSort*: cuántos intercambios de vecinos ordenan la lista.
4	Lógica, en papel: teorema, estrategia, desarrollo y conclusión; el invariante demostrado por inducción, la demostración directa y la por contradicción; la cota con O exhibiendo los testigos c y k; el invariante de la bisección de UVa 11413.	

Nivel de los ejercicios

El nivel de referencia son las tareas del curso de Estructuras de Datos y la primera tarea de Árboles y Grafos. Los problemas de clase son de ese nivel, y los tres de tarea de cada sesión están escalados:

- **Base.** el tema de la sesión aplicado directo.
- **Intermedio.** el tema más un formato de entrada o de salida que hay que respetar al pie de la letra.
- **Exigente.** el tema con una decisión de diseño adentro, sea la estructura o el costo.

Todos los problemas salen de UVa. La dirección del enunciado y la del envío van escritas completas, porque el ejercicio no termina hasta que el juez responde. Cada solución se entrega con su costo, analizado como en clase.

Sábado 17 de octubre. Estructuras y un recorrido

La sesión fue un repaso de Estructuras de Datos, con lo que cuesta cada operación, y resolvió un problema.

Hora 4. UVa 11078, *Open Credit System*.
https://vjudge.net/problem/UVA-11078
Envío: https://vjudge.net/problem/UVA-11078

Se busca el mayor valor de $a_i - a_j$ con $i < j$, donde el índice menor es el estudiante más antiguo. El parámetro es la posición j: para cada una, la mejor resta usa el máximo de $a_0, \ldots, a_{j-1}$, y ese máximo se actualiza al avanzar. Basta un recorrido. El doble ciclo revisa unos 5×10^9 pares en un caso con

$n = 100\,000$, y la entrada trae varios. La trampa está en la muestra: con una secuencia creciente la respuesta es -1, así que el máximo arranca en la primera resta y no en 0.

Costo. El tamaño es n, el número de estudiantes del caso. El ciclo corre $n - 1$ veces y cada vuelta hace dos comparaciones: $O(n)$ por caso, contra $O(n^2)$ del doble ciclo.

Sábado 24 de octubre. Tramos, matrices y lógica

Programación (tres horas)

Horas 1 y 2. UVa 507, *Jill Rides Again.*
https://vjudge.net/problem/UVA-507
Envío: https://vjudge.net/problem/UVA-507

La primera hora armó la idea con los acumulados del arreglo y la segunda llevó la solución al computador. Cada ruta trae $s - 1$ valores, uno por tramo entre paradas, y se pide el tramo continuo de mayor suma. El parámetro es el valor donde termina el tramo. El mejor que acaba ahí extiende al que acababa en el anterior, salvo cuando esa suma ya es negativa: entonces conviene empezar de nuevo. Se reinicia con suma negativa y no con cero, porque un prefijo de suma cero alarga el tramo sin cambiar su valor, y el enunciado desempata por el tramo más largo y después por el que empieza antes. Por eso el mejor se actualiza cuando la suma es mayor o cuando, con suma igual, el tramo es más largo. Las paradas se numeran desde 1: el tramo que va del valor i al j se imprime entre las paradas i y $j + 1$. Si la mejor suma no es positiva, la salida es `Route r has no nice parts`.

Costo. El tamaño es s, el número de paradas. Un solo ciclo recorre los $s - 1$ valores con trabajo constante en cada uno: $O(s)$ por ruta. Probar todos los tramos con dos ciclos sería $O(s^2)$.

Hora 3. UVa 108, *Maximum Sum.*
https://vjudge.net/problem/UVA-108
Envío: https://vjudge.net/problem/UVA-108

La matriz es de $N \times N$, con $N \leq 100$, y se pide el subrectángulo de mayor suma. Probar cada rectángulo con acumulados en dos dimensiones cuesta $O(N^4)$. Lo que baja el costo es fijar el par de filas, la de arriba y la de abajo. Con ellas fijas, la suma de cada columna entre las dos filas forma un arreglo de N valores, y el mejor rectángulo con esas filas es el mejor tramo de ese arreglo: el problema de las horas 1 y 2. Los acumulados por columna dan cada suma en $O(1)$. Hay dos trampas. Los N^2 números llegan repartidos en líneas cualesquiera, así que se leen de corrido sin contar líneas. Y si todo es negativo la respuesta es negativa, porque el rectángulo tiene al menos una celda.

Costo. El tamaño es N, el lado de la matriz. Los acumulados se llenan con dos ciclos anidados de N vueltas, $O(N^2)$. Los dos ciclos de las filas dan $N(N + 1)/2$ pares, y para cada par el tramo recorre N columnas: $O(N^3)$ en total. Con $N = 100$ son 5050 pares y unas 505 000 vueltas del ciclo interno.

Lógica (una hora)

La hora usó las diapositivas de lógica del 17 de octubre, que ese día no se dieron. Proposiciones y conectivos, y la implicación con su recíproca y su contrarrecíproca. Después, predicados y cuantificadores con su dominio y el rango de los índices. El ejercicio es traducir propiedades a fórmula, del mundo real (todo inscrito tiene nota, ningún horario se cruza con otro) y de los arreglos (el arreglo está ordenado, un valor aparece en un rango). Con eso se lee la definición que el curso usa desde la primera clase: $f \in O(g)$ cuando existen $c > 0$ y $k \geq 0$ tales que $f(n) \leq c\, g(n)$ para todo $n \geq k$.

Tarea

Base. UVa 100, *The 3n + 1 problem.*
https://vjudge.net/problem/UVA-100

Envío: https://vjudge.net/problem/UVA-100

Un ciclo aplica la regla hasta llegar a 1 y cuenta los números que imprime; el recorrido del rango se queda con el máximo. La longitud cuenta el 1, así que la de 22 es 16 y la de 1 es 1. La trampa es que i puede llegar mayor que j: el recorrido va del menor al mayor, pero la salida repite i y j en el orden en que llegaron. Los valores intermedios pasan de 10 000, así que no sirven como posición de un arreglo del tamaño de la entrada.

Costo. Hay dos ciclos anidados. El externo corre una vez por cada número del rango, $b - a + 1$ veces; el interno, tantas veces como pasos tenga el ciclo de ese número. Si L es la longitud más larga del rango, el total por pareja es $O((b - a + 1)\, L)$.

Intermedio. UVa 10189, *Minesweeper.*
https://vjudge.net/problem/UVA-10189
Envío: https://vjudge.net/problem/UVA-10189

Por cada celda sin mina se cuentan las minas de sus ocho vecinas, con los dos desplazamientos recorriendo $-1, 0, 1$ y la comprobación de borde. El primer número son las filas y el segundo las columnas. Las trampas están en los bordes y en la salida: las esquinas tienen menos de ocho vecinas y un campo de 1×1 no tiene ninguna; la línea en blanco va entre campos y no después del último; y las minas se conservan tal como llegaron. El par `0 0` cierra la entrada y no se procesa.

Costo. Con n filas y m columnas, los dos ciclos de las celdas corren nm veces y los dos de los desplazamientos, 9 veces por celda: $O(nm)$ por campo.

Exigente. UVa 620, *Cellular Structure.*
https://vjudge.net/problem/UVA-620
Envío: https://vjudge.net/problem/UVA-620

Una cadena sana es `A`, o una sana seguida de `AB`, o `B` seguida de una sana y de `A`. La definición ya es una recursión, y se escribe por índices: la función recibe el rango $[ini, fin]$ de la cadena y nunca copia una subcadena. El caso base es la cadena de una letra, y los dos casos recursivos llaman sobre el rango sin las dos letras del final o sin la primera y la última. Los extremos deciden la rama: la forma crecida termina en `B` y la mutagénica en `A`, así que mirar la primera y la última letra basta. La trampa está en el interior de esas dos formas, que tiene que ser una cadena sana y no vacía: `AB` y `BA` parecen formas válidas y son `MUTANT`. Contar letras no sirve, porque el interior se clasifica completo. En la muestra, `BAABA` es `MUTAGENIC` porque `AAB` es sana, y `BAAB` es `MUTANT` porque termina en `B` y `BA` no es sana.

Costo. Sobre una cadena de n letras, cada llamada quita dos letras y hace a lo más una llamada, así que hay a lo más $(n + 1)/2$ llamadas, cada una con trabajo constante: $O(n)$. Copiar la subcadena en cada llamada cuesta $n + (n - 2) + (n - 4) + \cdots$, que es $\Theta(n^2)$.

Sábado 7 de noviembre. Bisección y recorridos en grafos

Programación (tres horas)

Hora 1. UVa 12032, *The Monkey and the Oiled Bamboo.*
https://vjudge.net/problem/UVA-12032
Actividad: https://cardel.github.io/notasUniversidad/actividades/ayg-puj/busqueda-binaria/escalera.html

Una escalera tiene n peldaños a alturas $r_1 < \ldots < r_n$ medidas desde el suelo, y se sube de uno en uno sin saltarse ninguno. Se arranca con un factor de fuerza k: ningún salto puede medir más de k, y el salto que mide exactamente k baja el factor en uno. Se pide el menor k con el que se llega arriba.

Buscar el mínimo directamente obliga a razonar sobre todos los saltos a la vez. La pregunta que sí se contesta de una pasada es la otra: *¿se llega arriba empezando con fuerza k?* Se simula la subida llevando la fuerza actual, cada salto se compara contra ella y el que la iguala la gasta en uno. Esa función es monótona, porque con $k + 1$ la fuerza disponible es siempre mayor o igual y ningún salto que cabía deja

de caber, de modo que los candidatos quedan partidos en dos tramos y lo que se busca es el borde. Los límites salen del enunciado: por debajo el salto más grande, que hay que dar, y por arriba ese mismo más n, porque cada salto baja la fuerza a lo sumo en uno.

Costo. Cada vuelta de la bisección recorre los saltos una vez, y hay logarítmicas vueltas en el rango de k: $\Theta(n \log(\max s_i + n))$ por caso.

Hora 2. UVa 336, *A Node Too Far*.
https://vjudge.net/problem/UVA-336
Actividad: https://cardel.github.io/notasUniversidad/actividades/ayg-puj/representaciones-y-recorridos/tope.html

Cada mensaje de una red lleva un campo TTL. La estación que lo recibe le resta uno y lo reenvía solo si el resultado no es cero, de modo que el mensaje llega a los nodos a distancia menor o igual al TTL. Por cada consulta `nodo TTL` se pide cuántos nodos quedan por fuera.

El parámetro es el TTL, un tope de pasos. El recorrido en amplitud visita los nodos por distancia creciente: la primera vez que toca un nodo su distancia ya es la mínima, así que basta con no expandir los que llegaron a distancia TTL. El nodo de partida cuenta como alcanzado, incluso con TTL 0. Los números de los nodos pueden ser cualquier entero positivo, y por eso hace falta un diccionario de número a índice antes de armar la lista de adyacencia. Las trampas son tres. Expandir con $dist[u] \le TTL$ da un salto de más: la muestra responde 1, 0, 5, 0, 1, 0 en lugar de 5, 1, 8, 5, 3, 1. El contador de casos no se reinicia con cada red. Y el PDF no descarta una consulta con un nodo ausente, que no alcanza a ninguno, así que la respuesta es el total de la red. La salida dice `nodes` también cuando es uno solo.

Costo. $O(V + E)$ por consulta, con $V \le 30$ nodos y $E \le 435$ conexiones.

Hora 3. UVa 11686, *Pick up sticks*.
https://vjudge.net/problem/UVA-11686
Actividad: https://cardel.github.io/notasUniversidad/actividades/ayg-puj/orden-topologico-dfs-low/palitos.html

Hay n palitos y m pares `a b`, cada uno con el significado de que `a` está encima de `b`, con $n, m \le 1\,000\,000$. Un palito se recoge cuando ya se recogieron todos los que tiene encima. Se pide un orden de recogida o `IMPOSSIBLE`.

Cada nodo pasa por tres estados: sin visitar, en el camino actual y terminado. Una arista hacia un nodo en camino cierra un ciclo, porque desde ese nodo ya se había bajado hasta el actual. Una arista hacia un nodo terminado no cierra nada: ese nodo ya revisó todo lo que sale de él. Cuando un nodo termina, todo lo que sale de él ya terminó, de modo que el orden de terminación al revés respeta todas las aristas. La primera trampa es llevar solo dos estados, porque un nodo ya terminado parece un ciclo y el diamante $1 \to 2$, $1 \to 3$, $2 \to 4$, $3 \to 4$ sale `IMPOSSIBLE`. La segunda es la profundidad: una cadena de un millón de palitos revienta la pila de llamadas.

Costo. $\Theta(n + m)$ por caso. Medida con `ulimit -s` de 8192 KB, la recursiva de C++ revienta con 87 826 nodos de profundidad, así que al juez se envía la versión con pila explícita. La recursiva queda para explicar el algoritmo.

Lógica (una hora)

Invariantes de ciclo escritos como fórmula con cuantificador, índices y rango, y demostrados en tres pasos: inicialización, estabilidad y terminación. El primer ciclo es el de UVa 11078, que recorre $j = 1, \ldots, n-1$ con las variables *mayor* y *mejor*. Al empezar la vuelta j, con $1 \le j \le n$:

$$mayor = \max\{a_i : 0 \le i < j\} \ \wedge \ mejor = \max(\{a_0 - a_1\} \cup \{a_i - a_k : 0 \le i < k < j\}).$$

Al salir, con $j = n$, la segunda parte dice que *mejor* es la respuesta. En clase se demuestra la parte de *mayor*; de *mejor* se ven el candidato que falla, el invariante y la mejor pareja nueva, que muestra por qué depende de *mayor*. Su demostración va en las soluciones.

El ejercicio en papel es el ciclo de UVa 507 sin el desempate, sobre los valores $v_0, \ldots, v_{m-1}$. Al empezar la vuelta k, con $0 \le k \le m$, *suma* es la mayor suma de un tramo que termina en $k-1$, o 0 si todas son negativas, y *mejor* es la mayor suma de un tramo dentro de $v_0, \ldots, v_{k-1}$, o 0:

$$\mathit{suma} = \max\Big(\{0\} \cup \Big\{\textstyle\sum_{t=i}^{k-1} v_t : 0 \le i < k\Big\}\Big) \;\wedge\; \mathit{mejor} = \max\Big(\{0\} \cup \Big\{\textstyle\sum_{t=i}^{l} v_t : 0 \le i \le l < k\Big\}\Big).$$

La estabilidad es el paso de Kadane: el mejor tramo que termina en k es v_k solo o v_k pegado al mejor que termina en $k-1$, y el reinicio en 0 elige entre los dos.

Tarea

Base. UVa 10935, *Throwing cards away I.*
https://vjudge.net/problem/UVA-10935

El mazo es una cola, con la carta de arriba al frente. Mientras queden al menos dos cartas, la del frente se descarta y la siguiente pasa al final; la que queda es la respuesta. Las trampas están en la salida. Las cartas descartadas van separadas por coma y espacio, y ninguna línea lleva espacios al principio ni al final: con $n = 1$ no se descarta nada, y la primera línea es `Discarded cards:` sin espacio después de los dos puntos. La entrada termina con 0, que no se procesa.

Costo. Con n cartas, cada vuelta descarta una y el ciclo corre $n-1$ veces con trabajo constante en una cola: $O(n)$ por caso. Con una lista de Python sacando del frente, cada vuelta cuesta $\Theta(n)$ y el total sube a $O(n^2)$; con $n \le 50$ las dos pasan.

Intermedio. UVa 11286, *Conformity.*
https://vjudge.net/problem/UVA-11286

Cada estudiante elige cinco cursos, y la combinación es el conjunto, no el orden en que llegan: la primera y la tercera línea de la muestra son la misma. La clave es la lista de cursos ordenada, y un diccionario de clave a número de estudiantes da la popularidad de cada combinación. La trampa está en lo que se imprime. No es la mayor popularidad ni el número de combinaciones, sino el total de estudiantes que toman alguna combinación de popularidad máxima: si todas son distintas, como en el segundo caso de la muestra, la respuesta es el número de estudiantes. La entrada termina con 0.

Costo. Con n estudiantes, ordenar cinco cursos es trabajo constante y el diccionario responde en $O(1)$ promedio: $O(n)$ por caso.

Exigente. UVa 699, *The Falling Leaves.*
https://vjudge.net/problem/UVA-699

Cada nodo deja caer tantas hojas como su valor, y el hijo izquierdo queda una unidad a la izquierda de su padre y el derecho una a la derecha; se piden las pilas de hojas de izquierda a derecha. La entrada describe el árbol en preorden, con -1 para un subárbol vacío, así que la lectura ya es una recursión: la función lee un valor y, si no es -1, lo suma a la pila de su posición y se llama para el hijo izquierdo en la posición menos uno y para el derecho en la posición más uno. El árbol no se construye; basta la posición. La decisión de diseño está en guardar las pilas: con la raíz en la posición 0, los nodos de la izquierda caen en posiciones negativas, y sirve un diccionario o un arreglo con la raíz en el centro. Las trampas son de lectura y de salida. Un árbol puede ocupar varias líneas, como el segundo de la muestra, así que los números se leen de corrido; la entrada termina con un -1 solo; y cada caso lleva `Case k:`, la línea de pilas separadas por un espacio y una línea en blanco, también después del último.

Costo. Con n nodos hay una llamada por nodo y una por cada uno de los $n+1$ subárboles vacíos, cada una con trabajo constante: $O(n)$ por árbol, más el recorrido de las pilas para imprimirlas.

Sábado 14 de noviembre. Colas de prioridad, bisección y dividir y vencerás

Programación (tres horas)

Hora 1. UVa 1203, *Argus*.
https://vjudge.net/problem/UVA-1203

Cada consulta registrada responde a los P segundos de registrarse y después cada P segundos; se piden las primeras K respuestas. La cola de prioridad guarda un par por consulta, el instante de su próxima respuesta y su número. Sale la de menor par, y vuelve a entrar con el instante aumentado en su periodo. El orden de los pares resuelve por sí solo el desempate que pide el enunciado: si dos consultas responden en el mismo instante, sale primero la de número menor. Las trampas son de lectura. La primera respuesta llega a los P segundos, no en el instante 0; el registro termina en una línea con `#`; y K llega a 10 000 con a lo más 1000 consultas.

Costo. Con q consultas, la cola nunca pasa de q pares. Llenarla son q inserciones y el ciclo de salida corre K veces, con una extracción y una inserción por vuelta, cada una en $O(\log q)$: el total es $O((q+K)\log q)$.

Hora 2. UVa 10341, *Solve It*.
https://vjudge.net/problem/UVA-10341
Actividad: https://cardel.github.io/notasUniversidad/actividades/ayg-puj/busqueda-binaria/resolver.html

Dados seis enteros con $0 \le p, r \le 20$ y $-20 \le q, s, t \le 0$, se busca x en $[0,1]$ que cumpla $pe^{-x} + q\sin x + r\cos x + s\tan x + tx^2 + u = 0$, con cuatro decimales, o `No solution` si no hay.

Aquí la función viene dada y quedan por mirarle dos cosas. La monotonía se comprueba derivando: en $f'(x) = -pe^{-x} + q\cos x - r\sin x + s\sec^2 x + 2tx$ cada sumando es menor o igual que cero en $[0,1]$ por los rangos del enunciado, de modo que f no crece y hay a lo sumo una raíz. Los extremos deciden si existe: con f decreciente hay raíz exactamente cuando $f(0) \ge 0 \ge f(1)$. Esa es la trampa del problema, porque sin esa comprobación el ciclo igual termina y devuelve un número que no es raíz de nada. Cien vueltas de bisección dejan el intervalo en 2^{-100} del original, mucho más fino que los cuatro decimales, y evitan discutir con qué tolerancia parar.

Costo. Cien evaluaciones por caso, o sea constante; con hasta 2100 líneas de entrada el total es lineal en la cantidad de casos.

Hora 2. UVa 11413, *Fill the Containers*.
https://vjudge.net/problem/UVA-11413

La explicación recuerda la búsqueda binaria de la primera posición que cumple una condición, con `bisect` en Python y `lower_bound` en C++: funciona porque la condición es falsa en un prefijo y verdadera en el resto. La bisección sobre la respuesta usa la misma idea sobre las capacidades. Los recipientes de la banda se vacían en orden en m contenedores, cada recipiente entero en uno solo, y se pide la menor capacidad posible del contenedor más grande. En vez de buscar el óptimo se pregunta si alcanza con capacidad t. Un recorrido voraz lo responde: llena el contenedor actual mientras quepa el recipiente siguiente, abre otro cuando no cabe, y t alcanza si usa a lo más m. Si t alcanza, cualquier capacidad mayor también, y esa monotonía es la que hace funcionar la bisección entre el recipiente más grande y la suma de todos. La trampa está en los rangos: m llega a 10^6 con a lo más 1000 recipientes, así que pueden sobrar contenedores, y la cota inferior de la bisección es el recipiente más grande, porque ninguno se divide. Hay varios casos hasta fin de archivo.

Costo. Con n recipientes que suman S, el recorrido voraz corre n veces y la bisección sobre $[\text{máx}, S]$ corre $O(\log S)$ veces: el total es $O(n \log S)$, y no depende de m.

Hora 3. UVa 10810, *Ultra-QuickSort*.
https://vjudge.net/problem/UVA-10810

Actividad: https://cardel.github.io/notasUniversidad/actividades/ayg-puj/recursion/inversiones.html

Dada una lista de $n < 500\,000$ enteros distintos, se pide cuántos intercambios de elementos vecinos hacen falta para dejarla ordenada. Como cada intercambio de vecinos arregla exactamente una pareja fuera de orden, lo que se pide es el número de parejas (i, j) con $i < j$ y $a_i > a_j$, y contarlas una por una cuesta $\Theta(n^2)$.

Las parejas de un tramo $[ini, fin)$ se reparten en tres clases que no se pisan: las dos puntas en la mitad izquierda, las dos en la derecha, y una en cada mitad. Las dos primeras son el mismo problema sobre la mitad del tamaño, y ahí está la recursión; el caso base es un tramo de un elemento, que no tiene ninguna pareja. La tercera clase es lo que hay que combinar, y se cuenta gratis mientras se mezclan las dos mitades ya ordenadas: cuando sale un elemento de la derecha, todos los que siguen pendientes en la izquierda son mayores que él, así que aporta tantas parejas como elementos le falten a la izquierda. Que las mitades queden ordenadas es lo que permite contar esa clase en una sola pasada. La función trabaja por índices sobre el mismo arreglo, y el auxiliar de la mezcla se reserva una vez por caso.

Costo. Hay $\log_2 n$ niveles de llamadas y en cada uno la mezcla mueve los n elementos una vez, de modo que el total es $\Theta(n \log n)$. El contador va en `long long`, porque en el peor caso la respuesta llega al orden de 10^{11}. Medido con $n = 499\,999$, la versión de C++ tarda 0,20 s y la de Python 5,1 s, así que al juez se envía la de C++.

Lógica (una hora)

Una demostración se escribe en cuatro partes: el teorema, la estrategia que se va a seguir, el desarrollo y la conclusión. La de un invariante es por inducción sobre las vueltas del ciclo: la inicialización es el caso base, la estabilidad es el paso inductivo y la terminación es lo que el invariante dice al salir. Con la misma forma se ven la demostración directa y la por contradicción, con un ejemplo sobre arreglos para cada una.

Después, una cota de O grande en cuatro partes, exhibiendo los testigos c y k, como en el ejercicio 6 de las diapositivas de lógica del 17 de octubre: $2n^2 + 3n \in O(n^2)$. Donde CLRS escribe n_0, aquí va k.

La hora cierra con el invariante de la bisección de UVa 11413. Con $c_0, \ldots, c_{n-1}$ los recipientes y S su suma, la búsqueda va sobre $[l, r]$, que empieza en $[\max_i c_i, S]$, y $alcanza(t)$ dice si el recorrido voraz usa a lo más m contenedores de capacidad t:

$$(\forall t \mid \max_i c_i \le t < l : \neg\, alcanza(t)) \;\wedge\; alcanza(r).$$

Al inicio el rango del cuantificador es vacío, y S alcanza porque un solo contenedor lo recibe todo. Cada vuelta prueba el punto medio t: si alcanza, r baja a t; si no, l sube a $t + 1$, y la monotonía mantiene las dos partes. El ciclo termina porque $r - l$ baja en cada vuelta, y al salir, con $l = r$, la capacidad l es la respuesta.

Tarea

Base. UVa 10954, *Add All*.
https://vjudge.net/problem/UVA-10954

Cada suma cuesta lo que suman sus dos operandos, así que conviene sumar siempre los dos menores. La cola de prioridad entrega esos dos en tiempo logarítmico y recibe de vuelta el resultado, que vuelve a competir con los demás. Por eso ordenar una vez y sumar en fila no alcanza: ahí está el ejercicio. La trampa es el acumulado: con $N = 5000$ números iguales a 99 999 el costo mínimo pasa de 6×10^9, más de lo que cabe en 32 bits. La entrada termina con $N = 0$, que no se procesa.

Costo. Con N números, el ciclo corre $N - 1$ veces, con dos extracciones y una inserción en $O(\log N)$ cada una: $O(N \log N)$.

Intermedio. UVa 10611, *The Playboy Chimp.*
https://vjudge.net/problem/UVA-10611

Las N alturas llegan en orden no decreciente, y para cada consulta q se pide la más alta menor que q y la más baja mayor que q. Una altura igual a q no sirve para ninguna de las dos. Dos búsquedas binarias dan los bordes: la primera posición con altura $\geq q$ y la primera con altura $> q$. La anterior a la primera es la más alta menor; la segunda es la más baja mayor. Lo que permite buscar así es el orden, porque cada condición es falsa en un prefijo del arreglo y verdadera en el resto. La trampa son los repetidos y los bordes: cuando una de las dos alturas no existe, se imprime una `X` mayúscula en su lugar, y la entrada trae un solo caso.

Costo. El tamaño son las N alturas y las Q consultas. Cada búsqueda parte el rango a la mitad en cada vuelta, así que su ciclo corre a lo más $\lfloor \log_2 N \rfloor + 1$ veces: $O(N + Q \log N)$ con la lectura. Recorrer el arreglo en cada consulta es $O(NQ)$, unos $1{,}25 \times 10^9$ pasos con $N = 50\,000$ y $Q = 25\,000$.

Exigente. UVa 11624, *Fire!*
https://vjudge.net/problem/UVA-11624

Joe y el fuego avanzan una casilla por minuto, en vertical o en horizontal, y Joe sale por cualquier casilla del borde. El parámetro son las $R \times C$ casillas, con $R, C \leq 1000$. Lo resuelven dos recorridos en amplitud. El primero arranca con todas las casillas `F` en la cola, con distancia 0, y da el minuto en que el fuego llega a cada casilla. El segundo parte de `J` y solo entra a una casilla si llega antes que el fuego. La trampa principal es esa comparación, que es estricta: en la segunda muestra Joe y el fuego llegan a la vez a las dos casillas libres, y con $\leq$ saldría 2 en lugar de `IMPOSSIBLE`. El enunciado asegura una sola `J`, pero no asegura que haya alguna `F`, y una casilla a la que el fuego nunca llega sigue libre para Joe, así que la distancia -1 no puede entrar a la comparación sin más. La respuesta cuenta el minuto de salir: es la distancia a la casilla del borde más uno, y Joe parado en el borde responde 1.

Costo. Cada recorrido visita cada casilla una vez y mira cuatro vecinas: $O(RC)$ por caso, con hasta 10^6 casillas.

Para seguir

Propuesto. UVa 924, *Spreading the News.*
https://vjudge.net/problem/UVA-924

Cada empleado cuenta la noticia a sus amigos al día siguiente de enterarse, y se pide el día en que más empleados se enteran por primera vez y cuántos son. El día en que un empleado se entera es su distancia a la fuente en el grafo de amistades, y el recorrido en amplitud asigna esas distancias por niveles: basta contar cuántos quedan en cada nivel. La red se lee una vez y sirve para todos los casos, así que las distancias se reinician con cada fuente. Las trampas son tres. Los amigos de la fuente se enteran el día 1; ante un empate en el tamaño gana el primer día; y si nadie más que la fuente se entera, la salida es un solo `0`.

Costo. Con V empleados y A amistades, el recorrido saca de la cola a cada empleado una vez y revisa una vez los amigos de cada uno: $O(V + A)$ por fuente, y $O(T\,(V + A))$ con T fuentes.

Lo que hay que traer

- Computador con Python 3 y un compilador de C++ instalados y probados antes de la sesión.
- Cuenta en el juez UVa, https://vjudge.net/.
- La tarea de la sesión anterior intentada. La que no compila sirve igual: es la que se revisa.
- Papel para la hora de lógica.

Bibliografía

- T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms*. Tercera edición, MIT Press, 2009. Capítulos 2 y 3: invariantes y O grande.
- K. Rosen. *Matemáticas Discretas y sus Aplicaciones*. McGraw-Hill. Lógica, tipos de demostración e inducción.
- R. Thareja. *Data Structures Using C*. Oxford University Press, 2018.
- S. Halim, F. Halim, S. Effendy. *Competitive Programming 4*, 2020. Problemas de juez por tema.